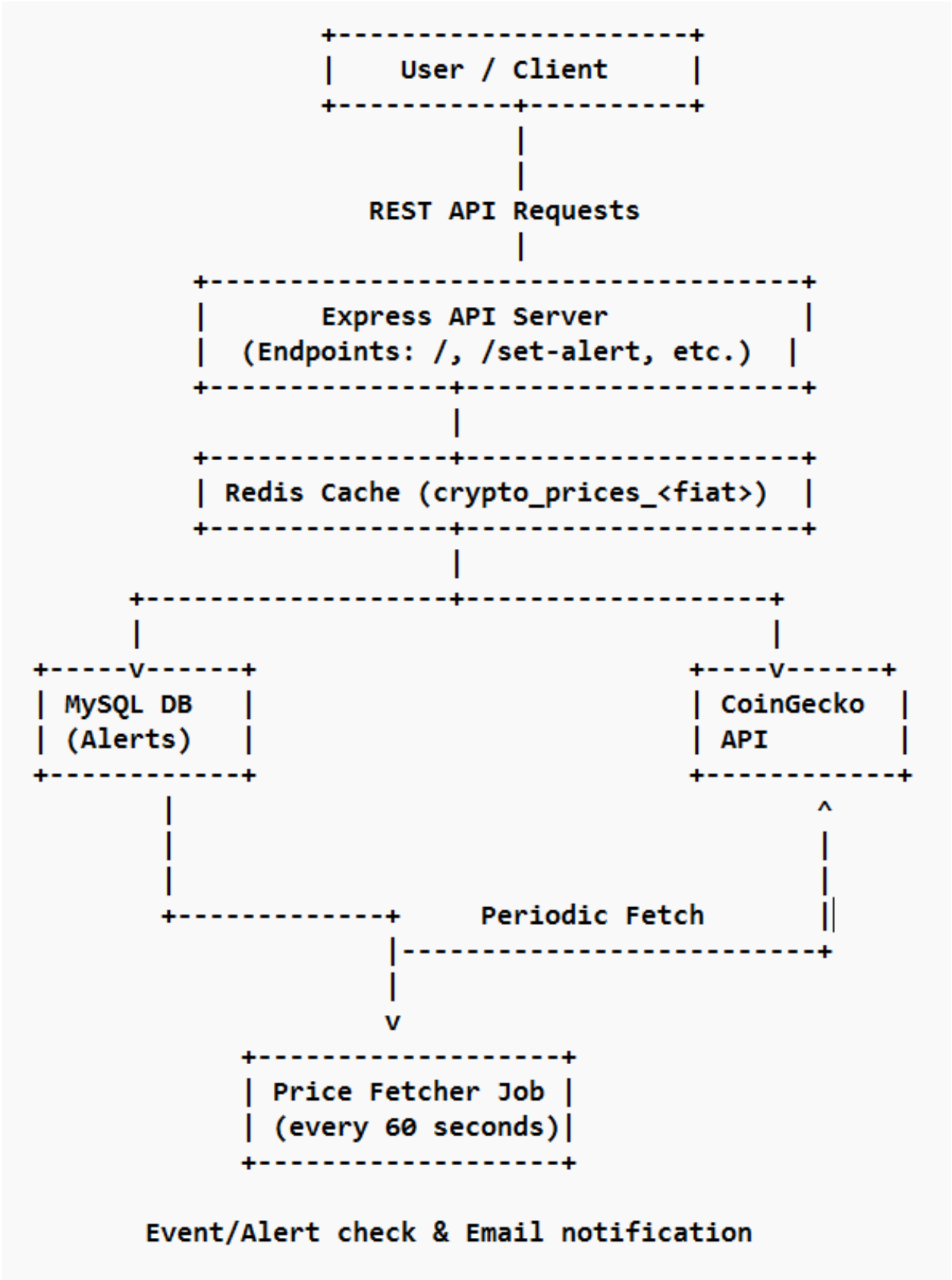
User / Client
REST API Requests
Express API Server
(Endpoints: /, /set-alert, etc.)
Redis Cache (crypto_prices_<fiat>)
MySQL DB
(Alerts)
CoinGecko
API
Periodic Fetch
Price Fetcher Job
(every 60 seconds)
Event/Alert check & Email notification

Detailed Flow with Explanation

1. User Interaction and Alert Setup

- Users interact via frontend or API clients.
- Users submit alert creation requests specifying:
- Cryptocurrency ID (e.g., bitcoin)
- Target price
- Alert condition ("above" or "below")
- Email address for notification
- Fiat currency (e.g., USD)
- The API server (/set-alert) validates input and stores these alerts persistently in the MySQL database.

2. Real-Time Cryptocurrency Price Fetching

- Every 60 seconds, a scheduled background job triggers the price fetcher.
- It queries CoinGecko's /coins/markets API for all cryptocurrencies, paginating results by market cap.
- The data includes price, market cap, 24h volume, and other metrics.
- Data is grouped by fiat currency to reduce the number of API calls.
- The fetched data is cached in Redis with a TTL (Time To Live) of 60 seconds.

3. Caching for Performance

- Redis acts as an in-memory cache for rapid price lookups.
- The cache keys are structured like crypto_prices_usd, holding JSON stringified price mappings.
- This reduces load on CoinGecko and accelerates user queries for current prices.

4. Alert Evaluation and Notification

- On each fetch cycle, alerts are retrieved from the MySQL database.
- Prices from Redis are compared against alert conditions:
- For "above" alerts, check if current price > target price.
- For "below" alerts, check if current price < target price.
- If triggered,
- An email notification is sent asynchronously using NodeMailer configured with Gmail SMTP.
- The alert is deleted to prevent duplicate notifications.

5. Exposed API Endpoints

- / - Returns paginated market data for all cryptocurrencies.
- /set-alert - Creates a new alert.

- /crypto-prices - Returns cached price data for a given fiat currency.

Challenges Faced & Their Solutions

Challenge	Solution
1.External API Rate Limiting	Batch requests by fiat currency and use pagination; cache results in Redis.
2.Ensuring Real-Time Data Accuracy	Poll every 60 seconds with cache expiry aligned to polling.
3.Handling Large Data Volume	Use CoinGecko pagination, only fetch needed data.
4.High Concurrency & Alert Notifications	Async email sending and atomic alert removal from DB to avoid duplication.
5.Missing Crypto Data (delisted or new)	Skip alerts with missing price data gracefully.
6.aching Consistency	Use Redis expiry and check cache before fetching from API.

Visual Diagram Notes for PDF

- System Architecture Diagram: Include the above text-based architecture with arrows and modify it with graphical blocks.
- Data Flow Chart: Show sequential steps in price fetching, caching, alert checking, and notification.
- Alert Trigger Flow: Flowchart how price compares to alert conditions leading to email notification and DB cleanup.
- Polling & Cache Expiry Timeline: Visualize the 60-second interval of data fetching, caching, and expiry coordination.